```
public static double sqrt(double a)
```

Για να υπολογίσετε την τετραγωνική ρίζα του 81, καλέστε

```
Math.sqrt(81)
```

Η συνάρτηση ceil επιστρέφει τη μικρότερη ακέραια τιμή που είναι μεγαλύτερη ή ίση με την παράμετρό της.

```
public static double ceil(double a)
```

Η κλήση ceil(9.1) επιστρέφει 10d, ενώ η κλήση ceil(9.0) επιστρέφει 9d.

Η συνάρτηση floor επιστρέφει τη μεγαλύτερη ακέραια τιμή που είναι μικρότερη ή ίση με την παράμετρό της.

```
public static double floor(double a)
```

Η κλήση floor(9.1) επιστρέφει 9d όπως και η κλήση floor(9.0).

Η συνάρτηση rint επιστρέφει την ακέραια τιμή που είναι πλησιέστερη στην τιμή της παραμέτρου της.

```
public static double rint(double a)
```

Η κλήση rint(9.4) επιστρέφει 9d και η κλήση rint(9.6) επιστρέφει 10d. Αν η τιμή της παραμέτρου βρίσκεται ακριβώς στη μέση μεταξύ δύο ακέραιων τιμών, τότε η rint επιστρέφει τον ακέραιο που είναι άρτιος. Επομένως, η κλήση rint(9.5) θα επιστρέψει 10d όπως ακριβώς και η κλήση rint(10.5).

Η συνάρτηση round επιστρέφει την ακέραια τιμή που είναι πλησιέστερη στην τιμή της παραμέτρου της. Η συνάρτηση είναι υπερφορτωμένη. Αν κληθεί με όρισμα double, επιστρέφει long και αν κληθεί με όρισμα float, επιστρέφει int. Αν η τιμή της παραμέτρου βρίσκεται ακριβώς στη μέση μεταξύ δύο ακέραιων τιμών, τότε η round επιστρέφει τον μεγαλύτερο από τους πλησιέστερους ακέραιους.

```
public static long round(double a)
public static int round(float a)
```

Η κλήση Math.round(9.4) επιστρέφει 9 και η κλήση Math.round(9.5) επιστρέφει 10.

Η συνάρτηση max επιστρέφει το μεγαλύτερο από τα ορίσματά της. Η συνάρτηση είναι υπερφορτωμένη και υποστηρίζει όλους τους αριθμητικούς τύπους. Οι διαθέσιμες διεπαφές έχουν ως εξής:

```
public static double max(double a, double b)
public static float max(float a, float b)
public static int max(int a, int b)
public static long max(long a, long b)
```

Αν καλέσετε την max με ορίσματα διαφορετικού τύπου, τότε θα κληθεί η έκδοση που είναι συμβατή και με τους δύο τύπους. Για παράδειγμα, αν την καλέσετε με double και long, τότε θα κληθεί η max(double, double). Έτσι η έκφραση

```
double m=Math.max(1d, 2L);
```

```
είναι σωστή. Ενώ η έκφραση
```

```
long m=Math.max(1d, 2L);
```

είναι λάθος.

Ακριβώς ανάλογη με την max είναι η min.

Η συνάρτηση abs επιστρέφει την απόλυτη τιμή της παραμέτρου της. Η abs είναι επίσης υπερφορτωμένη για όλους τους αριθμητικούς τύπους. Οι διαθέσιμες διεπαφές έχουν ως εξής:

```
public static double abs(double a)
public static float abs(float a)
public static long abs(long a)
public static int abs(int a)
```

7.6.2 Οι κλάσεις των θεμελιωδών τύπων

Έχουμε ήδη εξηγήσει πως για κάθε θεμελιώδη τύπο, η Java παρέχει μια αντίστοιχη κλάση. Κάθε μια τέτοια κλάση παρέχει σταθερές και συναρτήσεις που είναι χρήσιμες κατά την εκτέλεση εργασιών με θεμελιώδεις τύπους. Σε αυτήν την ενότητα συγκεντρώνουμε τις σημαντικότερες στατικές συναρτήσεις αυτών των κλάσεων.

7.6.2.1 Η κλάση Character

Η συνάρτηση isDigit επιστρέφει true αν η παράμετρος της αναπαριστά ψηφίο σύμφωνα με την κωδικοποίηση Unicode. Επομένως, εκτός από τα ψηφία που χρησιμοποιούνται στη Δυτική γραφή, υποστηρίζονται και τα ψηφία από διάφορους πολιτισμούς. Για παράδειγμα, οι χαρακτήρες ‘٢’, ‘٣’ και ‘٤’ αποτελούν ψηφία στο αραβικό σύστημα αρίθμησης. Η διεπαφή της συνάρτησης είναι

```
public static boolean isDigit(char ch)
```

Η συνάρτηση isLetter επιστρέφει true αν η παράμετρος της αναπαριστά κάποιο γράμμα σύμφωνα με την κωδικοποίηση Unicode. Προφανώς, η συνάρτηση δεν είναι αληθής μόνο για τους λατινικούς χαρακτήρες. Για παράδειγμα, οι χαρακτήρες ‘暘’, ‘暲’ και ‘晴’ αναπαριστούν γράμματα στη μανδαρίνικη γραφή. Η διεπαφή της συνάρτησης είναι

public static boolean isLetter(char ch)

Η συνάρτηση isLowerCase επιστρέφει true αν η παράμετρος της αναπαριστά πεζό γράμμα.

public static boolean isLowerCase(char ch)

Η συνάρτηση isSpaceChar επιστρέφει true αν η παράμετρος της αναπαριστά χαρακτήρα που είναι διαχωριστικό διαστήματος, γραμμής ή παραγράφου. Υπάρχουν συνολικά 19 τέτοιοι χαρακτήρες στο Unicode με κωδικούς 32, 160, 5760, 8192, 8193, 8194, 8195, 8196, 8197, 8198, 8199, 8200, 8201, 8202, 8232, 8233, 8239, 8287 και 12288. Η διεπαφή της συνάρτησης είναι

public static boolean isSpaceChar(char ch)

Η συνάρτηση isUpperCase επιστρέφει true αν η παράμετρος της αναπαριστά κεφαλαίο γράμμα. Η διεπαφή της συνάρτησης είναι

public static boolean isUpperCase(char ch)

Η συνάρτηση toString επιστρέφει την παράμετρό της ως String μήκους 1. Η διεπαφή της συνάρτησης είναι

public static String toString(char c)

7.6.2.2 Οι κλάσεις Integer, Double και Boolean

Καταρχάς να επισημάνουμε πως ό,τι αναφέρουμε σε αυτήν την ενότητα για την κλάση Integer ισχύει κατ' αναλογία και για τις κλάσεις Byte, Short και Long. Επίσης ό,τι αναφέρεται για την κλάση Double ισχύει κατ' αναλογία και για την κλάση Float.

Οι συναρτήσεις

```
public static int parseInt(String s) throws
NumberFormatException
public static double parseDouble(String s) throws
NumberFormatException
public static boolean parseBoolean(String s)
```

ανήκουν στις κλάσεις Integer, Double και Boolean, αντίστοιχα και μετατρέπουν το όρισμά τους σε int, double και boolean, αντίστοιχα.

Προσέξτε πως οι parseInt και parseDouble ενδέχεται να παράξουν λάθος τύπου NumberFormatException. Πρόκειται για λάθος στενά συγγενικό με το RuntimeException που είδαμε στην ενότητα 7.5. Αντίθετα, η parseBoolean δεν παράγει κανενός τύπου λάθος. Η διαφορά αυτή εξηγείται ως εξής: Η parseInt για παράδειγμα, για να λειτουργήσει σωστά θα πρέπει το όρισμά της να είναι μετατρέψιμο σε ακέραιο. Για παράδειγμα, η σειρά "*12.2" δεν μπορεί να μετατραπεί σε ακέραιο. Αν επομένως κληθεί η parseInt με αυτήν τη σειρά θα παραγάγει λάθος.

Αντίθετα η parseBoolean επιστρέφει true μόνο αν το όρισμά της είναι η σειρά "true" ανεξαρτήτως πεζών ή κεφαλαίων και false σε όλες τις άλλες περιπτώσεις.

Η συνάρτηση

```
public static int compare(double d1, double d2)
```

της κλάσης Double, συγκρίνει δύο πραγματικές μεταβλητές ή σταθερές και επιστρέφει 0 αν το d1 είναι αριθμητικά ίσο με το d2, τιμή μικρότερη του μηδενός αν το d1 είναι αριθμητικά μικρότερο από το d2, και τιμή μεγαλύτερη από το 0 αν το d1 είναι αριθμητικά μεγαλύτερο από το d2. Η συνάρτηση θεωρεί πως η τιμή Double.NaN είναι μεγαλύτερη από όλες τις άλλες τιμές τύπου Double και πως το θετικό 0d είναι μεγαλύτερο από το αρνητικό -0d. Διαφέρει η compare σε αυτό το σημείο σε σχέση με τους τελεστές ισότητας και ανισότητας <, <=, ==, >=, >. Για παράδειγμα ο κώδικας

```
System.out.println(Double.NaN>Double.MAX_VALUE);
System.out.println(Double.compare(Double.NaN,Double.MAX_VALUE
));
System.out.println(+0d>-0d);
System.out.println(Double.compare(+0d, -0d));
```

εμφανίζει

```
false
1
false
1
```

Επομένως ο τελεστής μεγαλύτερο από δεν βγάζει την Double.NaN μεγαλύτερη από την Double.MAX_VALUE ενώ η compare την βγάζει. Παρομοίως, ο τελεστής > δεν διαφοροποιεί μεταξύ των 2 μηδέν ενώ η compare διαφοροποιεί.

Οι συναρτήσεις

```
public static boolean isNaN(double v)
public static boolean isInfinite(double v)
```

επιστρέφουν true αν το όρισμά τους είναι Double.NaN ή Double.POSITIVE_INFINITY και Double.NEGATIVE_INFINITY, αντίστοιχα.

Τέλος, όλες οι κλάσεις των θεμελιωδών τύπων διαθέτουν τη στατική συνάρτηση toString παρόμοια με την toString της Character, ενότητα 7.6.2.1, που επιστρέφει την τιμή της παραμέτρου της με τη μορφή αλφαριθμητικής σειράς.

7.6.3 Η κλάση String

Η συνάρτηση

```
public static String copyValueOf(char[] data)
```

επιστρέφει ένα String που συντίθεται από τους χαρακτήρες του πίνακα data.
Η συνάρτηση

```
public static String copyValueOf(char[] data, int offset, int
count)
```

επιστρέφει ένα String που συντίθεται από count στον αριθμό χαρακτήρες του πίνακα data αρχίζοντας από τη θέση offset.

Οι συναρτήσεις valueOf επιστρέφουν την String αναπαράσταση του ορίσματός τους. Η valueOf είναι υπερφορτωμένη για όλους τους θεμελιώδεις τύπους. Ενδεικτικά παραθέτουμε τη διεπαφή της συνάρτησης για τον τύπο double

```
public static String valueOf(double d)
```

7.7 Προσδιοριστές προσπέλασης

Έχουμε συζητήσει αρκετά για τα στατικά μέλη της κλάσης. Εκείνο όμως που δεν έχουμε ξεκαθαρίσει είναι με ποια κριτήρια θα καθορίζουμε τον τύπο της πρόσβασης σε αυτά.

Δυστυχώς για αυτήν την απόφαση δεν υπάρχουν κατηγορηματικοί κανόνες και τα πάντα εξαρτώνται από τον στόχο που εξυπηρετεί η κάθε συνάρτηση. Ωστόσο, μια συζήτηση επάνω σε αυτό το θέμα μπορεί να μας βοηθήσει να λαμβάνουμε σωστότερες σχετικές αποφάσεις και να προστατεύουμε τους χρήστες των συναρτήσεών μας.

Η συζήτηση εδώ περιορίζεται στον έλεγχο της πρόσβασης σε στατικές συναρτήσεις και μεταβλητές.

Η Java υποστηρίζει 4 προσδιοριστές προσπέλασης, την ιδιωτική (private), τη δημόσια (public), την προσπέλαση πακέτου (package access) ή εξορισμού (default access) και την προστατευμένη (protected).

Για την προστατευμένη πρόσβαση δεν μπορούμε να πούμε και πολλά. Πρόκειται για τύπο πρόσβασης που σχετίζεται πολύ στενά με την κληρονομικότητα και παρουσιάζεται αναλυτικά στην ενότητα 13.1.2. Εκείνο όμως που μας ενδιαφέρει σχετικά με τα στατικά μέλη είναι πως παρότι ο μεταγλωττιστής δεν διαμαρτύρεται, σε γενικές γραμμές, η προστατευμένη πρόσβαση δεν ταιριάζει στα στατικά μέλη καθώς αυτά δεν υπακούουν στους κανόνες της κληρονομικότητας.

Πρόσβαση πακέτου δηλώνουμε όταν ένα μέλος της κλάσης θέλουμε να μπορεί να χρησιμοποιηθεί από όλες τις κλάσεις του ίδιου πακέτου αλλά όχι από κλάσεις που ανήκουν σε άλλα πακέτα. Θα μπορούσαμε να πούμε πως ό,τι είναι η ιδιωτική πρόσβαση σε επίπεδο κλάσης είναι και η πρόσβαση πακέτου σε επίπεδο πακέτου. Συχνά μάλιστα αναφέρεται ως ιδιωτική πρόσβαση πακέτου (package-private).

Ας δούμε τώρα ένα παράδειγμα με δύο συναρτήσεις όπου η μια είναι καλύτερο να δηλωθεί ως ιδιωτική και η άλλη ως δημόσια. Έστω λοιπόν πως θέλουμε να υλοποιήσουμε μια βιβλιοθήκη μαθηματικών συναρτήσεων μέσα στην οποία περιλαμβάνεται μια συνάρτηση που υπολογίζει έναν πραγματικό αριθμό υψωμένο σε ακέραια δύναμη. Για αυτό το παράδειγμα, εκπαιδευτικού χαρακτήρα, υποθέτουμε επίσης πως η συνάρτηση pow της Math δεν είναι διαθέσιμη,

Σε γενικές γραμμές είναι εύκολο να υλοποιήσουμε μια τέτοια συνάρτηση. Ας δούμε μια πρώτη προσέγγιση:

```
static double powerPos(double b, int e) { //Κώδικας 7.15
    if (e == 0) {
```

```
            return 1;
        }
        double rVal = 1;
        for (int i = 0; i < e; i++) {
            rVal *= b;
        }
        return rVal;
    }
```

Κώδικας 7.15 *Συνάρτηση που υπολογίζει την τιμή πραγματικού υψωμένου σε μη αρνητικό ακέραιο εκθέτη*

Στον κώδικα 7.15, η συνάρτηση powerPos επιστρέφει την τιμή της πρώτης παραμέτρου της υψωμένη στην τιμή της δεύτερης παραμέτρου της. Ωστόσο, η συνάρτηση επιστρέφει σωστά αποτελέσματα μόνο για μη αρνητικούς εκθέτες. Αν την καλέσετε για οποιονδήποτε αρνητικό εκθέτη, θα σας επιστρέψει λανθασμένα την τιμή 1. Αυτό μπορείτε να αντιληφθείτε εύκολα διαβάζοντας τον κώδικα 7.15. Εφόσον η if δεν θα οδηγήσει σε επιστροφή της συνάρτησης, η συνάρτηση θα προχωρήσει αρχικοποιώντας τη μεταβλητή rVal στο 1. Στη συνέχεια, η ροή δεν μπαίνει καθόλου στον βρόχο for μια και για την αρχική τιμή 0 του i δεν αληθεύει η i<e. Επομένως, η συνάρτηση καταλήγει να επιστρέφει 1.

Το γεγονός ότι η συνάρτηση μπορεί να παράγει λάθος για αρνητικούς εκθέτες την καθιστά εντελώς ακατάλληλη για δημόσια χρήση. Ωστόσο, αυτός που υλοποιεί την κλάση των μαθηματικών συναρτήσεων και γνωρίζει το συγκεκριμένο πρόβλημα θα μπορούσε να αξιοποιήσει τη συνάρτηση ώστε να διευκολυνθεί στην ανάπτυξη μιας πιο ολοκληρωμένης συνάρτησης υπολογισμού της δύναμης. Να τονίσουμε πάντως πως μια τέτοια συνάρτηση όπως η powerPos θα πρέπει να χρησιμοποιείται μόνο με πολλή προσοχή και πολύ περιορισμένα. Ας δούμε όμως πώς μπορεί να βοηθήσει η powerPos στην ανάπτυξη ορθής συνάρτησης υπολογισμού δύναμης.

```
    static double power(double b, int e) { // Κώδικας 7.16
        if (e >= 0) {
            return powerPos(b, e);
        }
        return 1 / powerPos(b, -e);
    }
```

Κώδικας 7.16 *Συνάρτηση που υπολογίζει την τιμή πραγματικού υψωμένου σε ακέραιο εκθέτη*

Όπως φαίνεται στον κώδικα 7.16, η συνάρτηση power ελέγχει την παράμετρο e που αντιπροσωπεύει τον εκθέτη. Αν η τιμή της είναι μη αρνητική, καλεί την powerPos και επιστρέφει σωστά τη δύναμη μη αρνητικού εκθέτη. Αν η τιμή της είναι αρνητική, δεν μπαίνει μέσα στην if, οπότε προχωρά και καλεί την powerPos με παράμετρο -e και επιστρέφει και πάλι σωστό αποτέλεσμα για αρνητικό εκθέτη.

Αυτό είναι ένα χαρακτηριστικό παράδειγμα όπου η μεν powerPos επιβάλλεται να δηλωθεί ως ιδιωτική ώστε να προστατεύεται ο χρήστης της βιβλιοθήκης μας από πιθανά λανθασμένη κλήση, ενώ η power μπορεί να δηλωθεί ως δημόσια ώστε να μπορεί ο χρήστης να υπολογίσει τις δυνάμεις που επιθυμεί.

7.8 Πακέτα

Ένα πακέτο συντίθεται ως ένα σύνολο κλάσεων ή και άλλων τύπων[6]. Επομένως, ένα πακέτο είναι ταυτόχρονα ένα σύνολο αρχείων πηγαίου κώδικα. Ο σκοπός του είναι να παρέχει έλεγχο πρόσβασης στα μέλη του και έναν μοναδικό ονοματοχώρο (namespace). Τα μέλη ενός πακέτου καθορίζονται από τη δήλωση που γίνεται υποχρεωτικά στην πρώτη γραμμή του αντίστοιχου αρχείου. Για τον έλεγχο πρόσβασης έχουμε ήδη μιλήσει στην ενότητα 7.7. Ας δούμε μερικές άλλες χρήσιμες λεπτομέρειες.

Η ανάπτυξη κώδικα στις μέρες μας είναι μια κατεξοχήν συνεργατική εργασία. Έτσι, οι προγραμματιστές που εργάζονται σε μία εταιρεία, συχνά χρησιμοποιούν κώδικες που έχουν αναπτύξει συνάδελφοί τους αλλά και κώδικες που έχουν αναπτυχθεί εκτός της εταιρείας. Προκύπτει εδώ ένα πρόβλημα. Τι θα συμβεί αν στον κώδικά μου έχω ορίσει μια συνάρτηση με ίδια ταυτότητα με μια συνάρτηση που έχει ορισθεί σε κώδικα τρίτου που όμως τον χρησιμοποιώ και εγώ; Για παράδειγμα, στον κώδικά μου χρησιμοποιώ τη συνάρτηση public static int

[6] Διεπαφών (interface), Απαριθμήσιμων τύπων (enumerated types) και τύπων σχολίων (annotation types).

f(int). Παράλληλα, σε έναν άλλο κώδικα που χρειάζομαι είναι ορισμένη συνάρτηση με την ίδια ταυτότητα σε ομώνυμη κλάση. Όταν καλώ π.χ. f(5), ποια από τις δύο f θα τρέξει; Τη λύση εδώ δίνει το πακέτο. Αν η δική μου f έχει ορισθεί στο πακέτο a και η άλλη f στο πακέτο b, τότε μπορώ να διαφοροποιήσω καλώντας a.ClassName.f(5) ή b.ClassName.f(5). Πώς όμως θα αποφύγουμε να έχουν και τα δύο πακέτα την ίδια ονομασία; Αυτό επιτυγχάνεται αποκλειστικά μέσω συμβάσεων για την ονοματολογία των πακέτων. Πιο συγκεκριμένα, ισχύουν οι ακόλουθες συμβάσεις:

1. Στην ονομασία ενός πακέτου θα πρέπει να χρησιμοποιούνται μόνο πεζοί χαρακτήρες ώστε να αποφεύγεται σύγκρουση με ονόματα κλάσεων ή άλλων τύπων.
2. Στην ονομασία ενός πακέτου πρέπει να συμμετέχει ένας τομέας διαδικτύου (internet domain). Οι επαγγελματίες συνηθίζουν να χρησιμοποιούν τη διεύθυνση email αντεστραμμένη όπως κάνουμε στους κώδικες αυτού του βιβλίου, π.χ. gr.ihu.cs.lmous.OOJ.k2.v2. Η οργάνωση των πακέτων και κλάσεων μέσα σε αυτά ώστε να μην προκύπτει σύγκρουση σε αναγνωριστικά είναι υπόθεση που εύκολα ελέγχεται σε συγκεκριμένο τομέα. Το θέμα είναι πως ένας τομέας δικτύου είναι υποχρεωτικά μοναδικός, οπότε έτσι εξασφαλίζεται και η μοναδικότητα στην ονομασία του πακέτου.
3. Σε πολλές περιπτώσεις, η επαναληπτική χρήση του ονόματος ενός πακέτου μέσα στον κώδικα είναι κουραστική. Έτσι η Java μας δίνει τη δυνατότητα να εισάγουμε διάφορα μέλη ενός πακέτου στον κώδικά μας και να τα χρησιμοποιούμε χωρίς την αναφορά πακέτου. Η εισαγωγή επιτυγχάνεται με τη χρήση της δεσμευμένης λέξης import.

Έτσι για παράδειγμα η πρόταση

```
import a.ClassName;
```

μας επιτρέπει να χρησιμοποιήσουμε τη συνάρτηση f της ClassName χωρίς την αναφορά πακέτου, δηλαδή ως ClassName.f(5).

Αν πάλι θέλουμε να εισάγουμε όλους τους τύπους από το πακέτο a, τότε θα πρέπει να χρησιμοποιήσουμε την import ως ακολούθως:

```
import a.*;
```

Προσοχή όμως, ο αστερίσκος δεν έχει το σύνηθες νόημα. Αν θέλουμε να εισάγουμε όλους τους τύπους του πακέτου a που το όνομά τους αρχίζει από B, η πρόταση

```
import a.B*;
```

δεν ισχύει και θα προκαλέσει λάθος μεταγλώττισης.

Επίσης, η πρόταση

```
import a.*;
```

εισάγει όλους τους τύπους του πακέτου a αλλά δεν εισάγει τίποτε από τα πακέτα a.b, a.c. Αν θέλουμε να εισάγουμε όλους τους τύπους του πακέτου a και όλους τους τύπους του πακέτου a.b, τότε πρέπει να χρησιμοποιήσουμε δύο import όπως φαίνεται παρακάτω.

```
import a.*;
import a.b.*;
```

7.9 Λυμένες ασκήσεις

Σε αυτήν την ενότητα παρουσιάζουμε μια σειρά από παραδείγματα στατικών συναρτήσεων ώστε να βοηθήσουμε τον αναγνώστη να εξοικειωθεί τόσο με την υλοποίηση όσο και με τη χρήση τους.

7.9.1 Μέγιστη τιμή πίνακα

Να υλοποιηθεί συνάρτηση που λαμβάνει ως παράμετρο έναν μονοδιάστατο πίνακα ακέραιων και επιστρέφει τη θέση της μέγιστης τιμής του.

Λύση
Καταρχάς θα πρέπει να αποσαφηνίσουμε τη διεπαφή της συνάρτησης. Σύμφωνα με την εκφώνηση, η συνάρτηση λαμβάνει έναν μονοδιάστατο πίνακα ακέραιων. Επομένως, η λίστα παραμέτρων είναι σαφής. Επίσης, σύμφωνα με την εκφώνηση, η συνάρτηση επιστρέφει το μέγιστο στοιχείο του πίνακα. Επομένως, επιστρέφει έναν ακέραιο. Ο κώδικας 7.17 παρουσιάζει τη συνάρτηση.

```
public static int max(int[] tbl) { //Κώδικας 7.17
    int max = tbl[0];
    int idx = 0;
    for (int i = 1; i < tbl.length; i++) {
        if (tbl[i] > max) {
            idx = i;
            max = tbl[i];
        }
    }
    return idx;
}
```

Κώδικας 7.17 *Συνάρτηση εύρεσης της μέγιστης τιμής σε πίνακα ακέραιων*

Για την υλοποίηση της συνάρτησης χρησιμοποιούμε δύο βοηθητικές μεταβλητές, την max και την idx. Η max χρησιμεύει για την αποθήκευση της τιμής του μεγαλύτερου στοιχείου και η idx για την αποθήκευση της θέσης του μεγαλύτερου στοιχείου. Η idx αρχικοποιείται ώστε να αναφέρεται στην αρχή του πίνακα και η max στην τιμή του στοιχείου στην θέση 0.

Στη συνέχεια, σε μια επαναληπτική διαδικασία ελέγχονται ένα προς ένα τα υπόλοιπα στοιχεία του πίνακα. Κάθε φορά που το υπό έλεγχο στοιχείο είναι μεγαλύτερο από την τιμή της max ενημερώνεται τόσο η max όσο και η idx. Στο τέλος του βρόχου for, η max είναι ενημερωμένη με την τιμή του μεγαλύτερου στοιχείου του πίνακα και η idx με τη θέση του.

Προσέξτε πως η for ξεκινάει από 1 και όχι από 0 όπως συνήθως. Αν ξεκινούσε από 0, στο πρώτο επαναληπτικό βήμα θα εξέταζε αν το στοιχείο στη θέση 0 είναι μεγαλύτερο από την τιμή της max. Η max όμως έχει αρχικοποιηθεί στην τιμή του στοιχείου στη θέση 0. Επομένως, πρόκειται για περιττή σύγκριση.

Επίσης, ενδιαφέρον έχει πως η συνάρτηση επιστρέφει τη θέση της μεγαλύτερης τιμής και όχι την ίδια την τιμή. Πράγματι, είναι πιο χρήσιμη η θέση παρά η τιμή καθώς αν έχω τη θέση, εύκολα μπορώ να προσπελάσω την τιμή.

Τέλος, ας σημειωθεί πως αν στον πίνακα υπάρχουν περισσότερα από ένα μέγιστα στοιχεία, η υλοποίηση του κώδικα 7.17 επιστρέφει τη θέση του πρώτου από τα μέγιστα.

7.9.2 Εξίσωση β' βαθμού

Να υλοποιηθεί συνάρτηση που επιστρέφει τη λύση εξίσωσης β' βαθμού.

Λύση
Καταρχάς θα πρέπει να δούμε πώς θα μοντελοποιήσουμε το πρόβλημα, δηλαδή στην προκειμένη περίπτωση να αποφασίσουμε ποια θα είναι η διεπαφή της συνάρτησης. Γνωρίζουμε από τα μαθηματικά πώς η εξίσωση β' βαθμού έχει τη μορφή

$$ax^2 + bx + c = 0, με\ a, b, c\ \in R\ και\ a \neq 0$$

Επομένως, η εξίσωση ορίζεται από τους συντελεστές της. Αυτούς πρέπει να περάσουμε παραμετρικά στη συνάρτηση. Τι όμως θα επιστρέφει η συνάρτηση; Όπως γνωρίζουμε, οι λύσεις εξαρτώνται από την τιμή της διακρίνουσας. Αν η διακρίνουσα, Δ, είναι μεγαλύτερη από το 0, τότε έχουμε δύο διακριτές πραγματικές λύσεις,

αν η διακρίνουσα είναι ίση με το 0, τότε έχουμε μια κοινή λύση και αν η διακρίνουσα είναι μικρότερη από το 0, η εξίσωση δεν έχει λύση.

Μία αντιμετώπιση είναι να επιστρέφουμε έναν πίνακα πραγματικών μεγέθους 2. Στην περίπτωση που $\Delta > 0$, τότε στην πρώτη θέση του πίνακα θα έχουμε τη μία ρίζα και στη δεύτερη την άλλη, αν $\Delta = 0$ και στις δύο θέσεις θα έχουμε την κοινή ρίζα, τέλος αν $\Delta < 0$, τότε και στις δύο θέσεις θα τοποθετήσουμε την τιμή Double.NaN.

Βέβαια όλα αυτά για να τα κάνουμε θα πρέπει να υπολογίσουμε τη διακρίνουσα. Παρόλο που ο χρήστης της συνάρτησης θα πληροφορηθεί για το πρόσημο της διακρίνουσας, δεν θα γνωρίζει την ακριβή τιμή της. Μία εναλλακτική προσέγγιση είναι η συνάρτησή μας να επιστρέφει πίνακα μήκους 3, όπου στη θέση 0 να τοποθετεί την τιμή της διακρίνουσας. Στον κώδικα 7.18 δίνεται αυτή η τελευταία εκδοχή της συνάρτησης.

```
static double[] triwnymo(double a, double b, double c) { // Κώδικας 7.18
    if (a == 0) {
        throw new RuntimeException();
    }
    double[] rVal = new double[3];
    double d = b * b - 4 * a * c;
    rVal[0] = d;
    if (d < 0) {
        rVal[1] = rVal[2] = Double.NaN;
        return rVal;
    }
    rVal[1] = (-b + Math.sqrt(d)) / (2 * a);
    rVal[2] = (-b - Math.sqrt(d)) / (2 * a);
    return rVal;
}
```

Κώδικας 7.18 *Συνάρτηση λύσης εξίσωσης β' βαθμού*

Σε αυτήν την περίπτωση η εξίσωση μπορεί να κληθεί όπως δείχνει ο κώδικας 7.19.

```
static void klisiTriwnymo() { // Κώδικας 7.19
    double[] solution = triwnymo(2, 5, -1);
    if (solution[0] < 0) {
        System.out.println("Η Εξίσωση σεν έχει λύση στο R");
    } else {
        System.out.println("χ1=" + solution[1] + " x2=" + solution[2]);
    }
}
```

Κώδικας 7.19 *Κλήση της συνάρτησης triwnymo*

Μία εναλλακτική κλήση δίνεται στον κώδικα 7.20

```
static void klisiTriwnymo2() { // Κώδικας 7.20
    double[] solution = triwnymo(2, 5, -1);
    if (Double.isNaN(solution[1])) {
        System.out.println("Η Εξίσωση σεν έχει λύση στο R");
    } else {
        System.out.println("x1=" + solution[1] + " x2=" + solution[2]);
    }
}
```

Κώδικας 7.20 *Εναλλακτική κλήση της συνάρτησης triwnymo*

Προσοχή όμως δεν είναι σωστή η κλήση

```
if (solution[1]==Double.NaN)….
```

καθώς εξορισμού η έκφραση Double.NaN==Double.NaN είναι ψευδής.

Σε μια εναλλακτική προσέγγιση μπορούμε να αναπαραστήσουμε το τριώνυμο ως έναν πίνακα πραγματικών μήκους 3. Η συνάρτηση μπορεί να υπερφορτωθεί εύκολα ώστε να υποστηρίζεται και αυτή η αναπαράσταση του τριωνύμου, όπως δείχνει ο κώδικας 7.21.

```
static double[] triwnymo(double[] syn) { // Κώδικας 7.21
    return triwnymo(syn[0], syn[1], syn[2]);
}
```

Κώδικας 7.21 *Υπερφόρτωση της συνάρτησης triwnymo*

Σε αυτήν την περίπτωση, γίνεται η αναπαράσταση εξισώσεων β' βαθμού ευκολότερη, όπως δείχνει ο κώδικας 7.22.

```
double[] ex1 = {2, 5, -1}, ex2 = {4, 4, -9};
double[] solution1 = triwnymo(ex1);
double[] solution2 = triwnymo(ex2);
```

Κώδικας 7.22 *Αναπαράσταση εξισώσεων β βαθμού ως double[]*

7.9.3 Fibonacci

Να αναπτυχθεί συνάρτηση που επιστρέφει τον ν-οστό όρο της ακολουθίας Fibonacci.

Λύση
Καταρχάς θα πρέπει να δούμε ποιος ακριβώς είναι ο ορισμός της ακολουθίας Fibonacci. Στο διαδίκτυο μπορούμε εύκολα να τον βρούμε. Αν συμβολίσουμε τον ν-οστό με Fn, τότε ο ορισμός της ακολουθίας έχει ως εξής:

$$n = 0 \rightarrow F_n = 0$$
$$n = 1 \rightarrow F_n = 1$$
$$n > 1 \rightarrow F_n = F_{n-1} + F_{n-2}$$

Επομένως, η συνάρτησή μας θα πρέπει να λαμβάνει μία ακέραια παράμετρο, μπορεί όμως να επιστρέφει ένα long, δεδομένου πως η τιμή του ν-οστού όρου είναι σημαντικά μεγαλύτερη από το n. Ο κώδικας 7.23 δίνει μια υλοποίηση της συνάρτησης.

```
static long fibonacci(int fibIdx) { //Κώδικας 7.23
    if (fibIdx < 0) {
        throw new RuntimeException();
    } else if (fibIdx <= 1) {
        return fibIdx;
    }
    int n = 2;
    long current = 1, previous = 1, beforePrevious;
    while (++n <= fibIdx) {
        beforePrevious = previous;
        previous = current;
        current = beforePrevious + previous;
    }
    return current;
}
```

Κώδικας 7.23 *Υλοποίηση της συνάρτησης Fibonacci*

Καθώς δεν υπάρχει αρνητικός όρος της ακολουθίας, αν η συνάρτηση κληθεί με παράμετρο μικρότερη του μηδενός, σημαίνει πως κάποιο λάθος έχει συμβεί. Διαφορετικά αν η παράμετρος fibIdx είναι μικρότερη ή ίση με το 1, δηλαδή αν είναι 0 ή 1, βάσει ορισμού, επιστρέφεται η ίδια η τιμή της παραμέτρου. Αν παρόλα αυτά δεν επιστρέψει η συνάρτηση σημαίνει ότι κλήθηκε με παράμετρο μεγαλύτερη του ένα, οπότε η τιμή του ν-οστού όρου πρέπει να υπολογιστεί ως το άθροισμα των δύο προηγούμενων όρων. Η μεταβλητή n αναπαριστά

την τάξη του όρου και ξεκινά από το 2, οι δε μεταβλητές current, previous και beforePrevious αναπαριστούν τις τιμές του ν-οστού όρου, του προηγούμενού του και του όρου πριν τον προηγούμενο του ν-οστού. Ο βρόχος while επισκέπτεται έναν-έναν τους όρους της ακολουθίας έως ότου φτάσει στον όρο που εκφράζει η παράμετρος fibIdx. Σε κάθε βήμα αυτής της επαναληπτικής διαδικασίας ενημερώνονται κατάλληλα οι προηγούμενοι όροι. Με την έξοδο από την while, η current είναι ενημερωμένη με την τιμή του όρου στη θέση fibIdx.

7.9.4 Λεκτικό αριθμού

Να υλοποιηθεί συνάρτηση static String lektiko(int num) που δέχεται μια ακέραια παράμετρο από 0 έως 999 και επιστρέφει τη διατύπωσή της στα Ελληνικά.

Λύση

Σύμφωνα με την εκφώνηση και με τη διεπαφή που δίνεται μια κλήση όπως η lektiko(125) θα πρέπει να επιστρέψει την αλφαριθμητική σειρά “εκατόν είκοσι πέντε”. Εδώ έχουμε δύο θέματα να μας απασχολήσουν. Πρώτον, πόσα και ποια λεκτικά μας είναι απαραίτητα για να σχηματίσουμε την απαιτούμενη διατύπωση όλων των αριθμών από 0 έως 999 και δεύτερον πώς θα εξάγουμε πόσες εκατοντάδες έχουμε, πόσες δεκάδες και πόσες μονάδες. Προσέξτε πως αν διαιρέσετε έναν ακέραιο με το 100 (ακέραια διαίρεση), τότε λαμβάνεται τον αριθμό εκατοντάδων που περιέχει. Αν στη συνέχεια βγάλετε τις εκατοντάδες, μένει το υπόλοιπο του ακέραιου. Για παράδειγμα: Έστω num=475, h ο αριθμός των εκατοντάδων και r το υπόλοιπο, τότε h=num/100=4, r=num-h*100=75. Στη συνέχεια διαιρούμε το υπόλοιπο με το 10 για να λάβουμε τον αριθμό των δεκάδων. Τέλος, οι μονάδες είναι το υπόλοιπο μετά την αφαίρεση των δεκάδων. Ο κώδικας 7.24 επιστρέφει το λεκτικό ενός ακέραιου από το 0 έως το 999.

```
static String lektiko(int num) { //Κώδικας 7.24
    if (num < 0 || num > 999) {
        throw new RuntimeException();
    }
    if (num == 0) {
        return "μηδέν";
    }
    int ekatontades = num / 100;
    num -= ekatontades * 100;
    int dekades = num / 10;
    num -= dekades * 10;
    String hundreds = "", tens = "", units = "";
    switch (ekatontades) {
        case 0:
            break;
        case 1:
            hundreds = "εκατόν";
            break;
        case 2:
            hundreds = "διακόσια";
            break;
        case 3:
            hundreds = "τριακόσια";
            break;
        case 4:
            hundreds = "τετρακόσια";
            break;
        case 5:
            hundreds = "πεντακόσια";
            break;
        case 6:
            hundreds = "εξακόσια";
            break;
        case 7:
```

```
            hundreds = "επτακόσια";
            break;
        case 8:
            hundreds = "οκτακοσια";
            break;
        case 9:
            hundreds = "εννιακόσια";
            break;
    }
    switch (dekades) {
        case 0:
            break;
        case 1:
            tens = "δέκα";
            break;
        case 2:
            tens = "είκοσι";
            break;
        case 3:
            tens = "τριάντα";
            break;
        case 4:
            tens = "σαράντα";
            break;
        case 5:
            tens = "πενήντα";
            break;
        case 6:
            tens = "εξήντα";
            break;
        case 7:
            tens = "εβδομήντα";
            break;
        case 8:
            tens = "ογδόντα";
            break;
        case 9:
            tens = "ενενήντα";
            break;
    }
    switch (num) {
        case 0:
            break;
        case 1:
            if (tens.equals("δέκα")) {
                tens = "έντεκα";
            } else {
                units = "ένα";
            }
            break;
        case 2:
            if (tens.equals("δέκα")) {
                tens = "δώδεκα";
            } else {
                units = "δύο";
            }
            break;
        case 3:
            units = "τρία";
            break;
        case 4:
```

```
                units = "τέσσερα";
                break;
            case 5:
                units = "πέντε";
                break;
            case 6:
                units = "έξη";
                break;
            case 7:
                units = "επτά";
                break;
            case 8:
                units = "οκτώ";
                break;
            case 9:
                units = "εννέα";
                break;
        }
        String rVal = hundreds + (tens != "" ? " " : "") + tens + (units != "" ?
" " : "") + units;
        //return rVal;
        return rVal.trim();
    }
```

Κώδικας 7.24 *Μετατροπή ακέραιου σε λεκτικό*

Προσέξτε την έκφραση tens!=””?” ”:””. Σημαίνει πως αν η μεταβλητή tens έχει πάρει τιμή διάφορη από ““, τότε επιστρέφουμε ένα διάστημα που θα διαχωρίσει τις εκατοντάδες από τις δεκάδες. Αν όμως η tens παραμένει ““, τότε σημαίνει πως δεν έχουμε δεκάδες, επομένως δεν υπάρχει λόγος να προσθέσουμε το αντίστοιχο διάστημα. Ίδια λογική ισχύει και για την units!=””?” ”:””.

7.9.5 Κλάση MyArrays

Να υλοποιηθεί η κλάση MyArrays που περιλαμβάνει τις ακόλουθες στατικές συναρτήσεις χωρίς να χρησιμοποιηθεί η κλάση Arrays.

```
public static int seqSearch(int[] t, int sE)
```

Πραγματοποιεί σειριακή αναζήτηση στον t για την τιμή sE. Επιστρέφει τη θέση στον t της sE εφόσον αυτή βρεθεί, διαφορετικά -1.

```
public static int seqSearch(String[] t, String sE)
```

Πραγματοποιεί σειριακή αναζήτηση στον t για την τιμή sE. Επιστρέφει τη θέση στον t της sE εφόσον αυτή βρεθεί, διαφορετικά -1.

```
public static int cntOccurances(int[] t, int sE)
```

Μετράει και επιστρέφει το πλήθος τιμών sE στον t.

```
public static String toString(int[] t)
```

Επιστρέφει μια String αναπαράσταση του t.

```
public static String toString(int[][] t)
```

Επιστρέφει μια String αναπαράσταση του t.

```
public static int[] reverse(int[] t)
```

Επιστρέφει έναν πίνακα ακέραιων που περιέχει τα στοιχεία του t με αντεστραμμένη διάταξη.

```
public static int sum(int[] t)
```

Επιστρέφει το άθροισμα των στοιχείων του t.

```
public static int max(int[] t)
```

Επιστρέφει τη θέση του μέγιστου στοιχείου του t.

```
public static boolean equals(int[] tA, int[] tB)
```

Επιστρέφει true αν κάθε στοιχείο του tA είναι και στοιχείο του tB και κάθε στοιχείο του tB είναι και στοιχείο του tA.

```
public static boolean equals(String[] tA, String[] tB)
```

Επιστρέφει true αν κάθε στοιχείο του tA είναι και στοιχείο του tB και κάθε στοιχείο του tB είναι και στοιχείο του tA.

```
public static int binarySearch(int[] t, int sE)
```

Πραγματοποιεί δυαδική αναζήτηση στον t για την τιμή sE. Αν η τιμή βρεθεί, επιστρέφει τη θέση της στον t, διαφορετικά επιστρέφει -1.

```
public static void swap(int[] t, int idx1, int idx2)
```

Αντιμεταθέτει το στοιχείο στη θέση idx1 του t με το στοιχείο στη θέση idx2.

```
public static int idxOfMin(int[] array, int startIdx)
```

Επιστρέφει τη θέση του ελάχιστου στοιχείου του t από τη θέση startIdx και μετά.

```
public static void sort(int[] t)
```

Ταξινομεί κατ'αύξουσα αριθμητική σειρά των πίνακα t.

Λύση
Ο κώδικας 7.25 παραθέτει μια λύση της άσκησης και ακολουθούν σχόλια.

```
public class MyArrays {

    public static int seqSearch(int[] t, int sE) {
        for (int i = 0; i < t.length; i++) {
            if (t[i] == sE) {
                return i;
            }
        }
        return -1;
    }

    public static int seqSearch(String[] t, String sE) {
        for (int i = 0; i < t.length; i++) {
            if (t[i].equals(sE)) {
```

```
                return i;
            }
        }
        return -1;
    }

    public static int cntOccurances(int[] t, int sE) {
        int cnt = 0;
        for (int e : t) {
            if (e == sE) {
                cnt++;
            }
        }
        return cnt;
    }

    public static String toString(int[] t) {
        if (t.length == 0) {
            return "[]";
        }
        String s = "[";
        for (int i = 0; i < t.length; i++) {
            s += t[i];
            if (i < t.length - 1) {
                s += ", ";
            }
        }
        s += "]";
        return s;
    }

    public static String toString(int[][] t) {
        if (t.length == 0) {
            return "[]";
        }
        String s = "[";
        for (int i = 0; i < t.length; i++) {
            s += toString(t[i]);
            if (i < t.length - 1) {
                s += ",\n";
            }
        }
        s += "]";
        return s;
    }

    public static int[] reverse(int[] input) {
        int[] rVal = new int[input.length];
        for (int i = 0; i < input.length; i++) {
            rVal[rVal.length - 1 - i] = input[i];
        }
        return rVal;
    }

    public static int sum(int[] t) {
        int sum = 0;
        for (int i : t) {
            sum += i;
        }
        return sum;
    }
```

```
public static int max(int[] t) {
    int max = t[0];
    int idx = 0;
    for (int i = 1; i < t.length; i++) {
        if (t[i] > max) {
            idx = i;
            max = t[i];
        }
    }
    return idx;
}

public static boolean equals(int[] tA, int[] tB) {
    if (tA.length != tB.length) {
        return false;
    }
    for (int cE : tA) {
        if (seqSearch(tB, cE) == -1) {
            return false;
        }
    }
    return true;
}

public static boolean equals(String[] tA, String[] tB) {
    if (tA.length != tB.length) {
        return false;
    }
    for (String cE : tA) {
        if (seqSearch(tB, cE) == -1) {
            return false;
        }
    }
    return true;
}

public static int binarySearch(int[] tbl, int schElement) {
    int from = 0, to = tbl.length - 1, mid = (to - from) / 2;
    while (to >= from) {
        if (tbl[mid] == schElement) {
            return mid;
        } else if (schElement < tbl[mid]) {
            to = mid - 1;
        } else {
            from = mid + 1;
        }
        mid = from + (to - from) / 2;
    }
    return -1;
}

public static void swap(int[] t, int idx1, int idx2) {
    int i = t[idx1];
    t[idx1] = t[idx2];
    t[idx2] = i;
}

public static int idxOfMin(int[] t, int startIdx) {
    int min = t[startIdx];
    int rVal = startIdx;
```